CS102: Data Structures and Algorithms

Week 4: Search Topologies & Sorting Contracts

Milestone 1: Search Topologies & Linear Iteration Space

In computer science, a search problem consists of locating an active target node situated within an indexed data collection frame. The fundamental mechanics of searching require evaluating structural configurations (sorted vs. unsorted collections) to optimize retrieval performance.

Lessons 1 & 2: The Linear Search Paradigm & Target-Collection Mechanics

A basic linear search traverses an unsorted collection sequentially from left to right, comparing each node directly against the target. The operational workload (the comparison step count) scales linearly at an asymptotic rate of O(n), because the loop's execution footprint is directly tied to the target's relative offset inside memory.

```
Target Node Search: 7 (Unsorted Data Footprint)
+---------+---------+---------+---------+---------+
| 14 | 3 | 8 | 7 | 22 |
+---------+---------+---------+---------+---------+
  Index 0 Index 1 Index 2 Index 3 Index 4

Step 1: Compare index 0 -> [14] == 7? False --> Move Right
Step 2: Compare index 1 -> [ 3] == 7? False --> Move Right
Step 3: Compare index 2 -> [ 8] == 7? False --> Move Right
Step 4: Compare index 3 -> [ 7] == 7? TRUE --> Exit Found!
```

■ Expanded Content: Structural Target Boundaries

- **Best-Case Complexity — O(1):** Target is positioned precisely at index 0. Operates in a single constant lookup check.
- **Worst-Case Complexity — O(n):** Target is situated at the final tail offset (index n-1) or is completely absent. Requires checking every node in the collection.

```
# Simulation: Sequential Linear Search with Explicit Operation Telemetry
def linear_search_telemetry(collection, target):
    comparisons = 0
    found_flag = False
    for item in collection:
        comparisons += 1
        if item == target:
            found_flag = True
            break
    return {"found": found_flag, "total_comparisons": comparisons}
```

Milestone 2: Asymptotic Search Scaling & Binary Splitting

As linear datasets expand toward enterprise scales, sequential scanning surfaces as a high-latency hazard. Shifting to sub-linear, logarithmic execution profiles requires enforcing pre-sorted memory invariants.

Lessons 3 & 4: Binary Search & The Logarithmic Halving Blueprint

The Binary Search algorithm executes a divide-and-conquer strategy over a pre-sorted array. By maintaining two pointer boundaries (low and high), it calculates a middle division offset (mid). Each unsuccessful comparison discards exactly 50% of the remaining search space, dropping time complexity down to a highly sustainable profile of O(log n).

```
Target Node Search: 15 (Pre-Sorted Array Frame, n = 7)

INITIALization STEP (Pass 1):
+---+---+---+----+----+----+----+
| 2 | 5 | 8 | 11 | 15 | 19 | 24 |
+---+---+---+----+----+----+----+
  ^ ^ ^
 Low Mid High Evaluate: array[3] -> 11 < 15? True
                                    Action: Low = mid + 1 (4)

HALVING EVALUATION (Pass 2):
xxxxxxxxxxxxx+----+----+----+
x (Discarded) x 15 | 19 | 24 |
xxxxxxxxxxxxx+----+----+----+
               ^ ^ ^
              Low Mid High Evaluate: array[5] -> 19 > 15? True
                                    Action: High = mid - 1 (4)

FINAL TARGET MATCH (Pass 3):
xxxxxxxxxxxxxxxxx+----+xxxxxxx
x (Discarded) x 15 | xxxxx
xxxxxxxxxxxxxxxxx+----+xxxxxxx
                   ^
              Low/Mid/High Evaluate: array[4] -> 15 == 15? TRUE
```

```python
# Safe Binary Search Implementation under Asymptotic O(log n) Performance
def binary_search_trace(sorted_data, target):
    low = 0
    high = len(sorted_data) - 1
    steps = 0
    while low <= high:
        mid = (low + high) // 2
        steps += 1
        if sorted_data[mid] == target:
            return {"index": mid, "executed_steps": steps}
        elif sorted_data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return {"index": -1, "executed_steps": steps}
```

Lessons 5 & 6: Master Reference Matrix — Linear vs. Logarithmic Performance

Collection Scale (n)	Linear Search Max Cost (O(n))	Binary Search Max Cost (O(log n))	Algorithmic Selection Rule
n = 10 Elements	10 Comparisons	~3–4 Comparisons	Linear Search: Simplest path for small unsorted arrays.
n = 1,000 Elements	1,000 Comparisons	~10 Comparisons	Binary Search: Clear performance win if data is pre-sorted.
n = 1,000,000 Elements	1,000,000 Comparisons	~20 Comparisons	`Mandatory Boundary: Binary search required for scale.`

Milestone 3: Quadratic Sorting Topologies & Stability

Sorting transitions data from chaotic sets into structured arrays, enabling advanced reporting and clean analytical summaries (e.g., Top-N slicing, medians, and outlier detection).

Lessons 7 & 8: Bubble Sort Mechanics & Asymptotic Work Proof

Bubble Sort operates via continuous neighbor-swapping. It scans the array from left to right, comparing adjacent elements and swapping them if they violate the sorted order. This forces the largest unsorted values to gradually 'bubble' rightward toward the tail end of the array. Total comparison operations are calculated as $(n^2 - n) / 2$, matching worst-case quadratic growth $O(n^2)$.

Lessons 9 & 10: Insertion Sort Mechanics & Adaptive Efficiency

Insertion Sort constructs an ordered boundary frame on the left side of the list, moving from left to right to isolate each unsorted value as a key. It then slides the key backward into its correct slot inside the sorted zone, shifting larger sorted elements to the right. While worst-case runs are $O(n^2)$, it scales down to a near-linear performance of $O(n)$ on pre-sorted data.

■ Expanded Content: The Structural Showdown (Why Shifting Beats Swapping)

- **Operation Cost:** Bubble sort requires swapping, costing 3 pointer mutations per check. Insertion sort utilizes shifting, requiring only 1 copy mutation, executing roughly 1/3 of the physical workload.
- **Adaptive Scanning:** On partially sorted lists, Insertion Sort stops inner loop traversal early upon reaching its sort-wall, while baseline Bubble Sort requires continuous nested passes.

```
# High-Instruction Sorting Implementations with Performance Telemetry
def benchmark_bubble_sort(nums):
    arr = list(nums)
    n = len(arr)
    comparisons, swaps = 0, 0
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
                swapped = True
        if not swapped: break
    return {"comparisons": comparisons, "swaps": swaps}

def benchmark_insertion_sort(nums):
    arr = list(nums)
    comparisons, shifts = 0, 0
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if arr[j] > key:
                arr[j + 1] = arr[j]
                shifts += 1
                j -= 1
            else: break
        arr[j + 1] = key
    return {"comparisons": comparisons, "shifts": shifts}
```

Lessons 13 & 14: Native Python Sorting & Composite Stable Contracts

Production applications rely on Python's built-in sorting tools (implementing Timsort). Python's sort is guaranteed to be stable, meaning elements with equal keys maintain their original relative order, allowing for clean, multi-pass composite sort sequences.

```
# Production Strategy: Multi-Criteria Leaderboard Sorting using Composite Keys
leaderboard_data = [
    {"name": "Ava", "score": 100, "level": 3},
    {"name": "Ben", "score": 150, "level": 2},
    {"name": "Cara", "score": 150, "level": 5}
]

# Sort by score descending, then by level descending, then by name ascending
calibrated_leaderboard = sorted(
    leaderboard_data,
    key=lambda item: (-item["score"], -item["level"], item["name"])
)
```